Adaptive Control Laboratory, Part I

Coupled DC motors: identification, MIT rule, MRAC, ADI, robust and angle control

Experimental report, session of 14 October 2026 (incomplete: motor failure during Lab I.6)

Status at a glance

Lab	Status	Main result
I.1 Identification	Done	FOPDT model + stick-slip: K ≈ 0.020 rad/s/PWM, τ = 0.128 s, Td = 0.115 s; breakaway 115–200 PWM, stop 60–75 PWM; PI tracks with zero error, P cannot overcome stiction; load pot max end: −75 % speed
I.2 MIT rule	Done	Experimental stability condition $\gamma \cdot r0^2 < c$, with c between 14.7 and 16
I.3 Scalar MRAC	Done	Stable 10 min, but linear parameter drift (no PE on a constant reference)
I.4 Scalar ADI	Done	Stable; stiction blocks the slow-sine half-periods; slower drift than MRAC
I.5 Robust MRAC/ADI	Done	Friction-aware models halve the error (MRAC sine 9.9 → 4.6 %, ADI 29 → 6.3 %); only σ-mod stops the ADI drift
I.6 Angle (vector MRAC)	Partial	3 of 12 runs valid (angle tracking works, ~0.35 rad RMS); motor failed during run 4. Robust comparison and long runs NOT done

All numbers come from the automatically generated reports in `results/rig/` (one folder per lab with summary.md, a CSV of every run and the plots). Velocities are in rad/s at the motor shaft, inputs in PWM counts (±255). In Labs I.2–I.6 the adaptive laws work with normalised signals $y = \omega/\omega_{max}$ and $u_n = u/255$. “e” is the normalised tracking error $y - y_m$, so 0.05 means 5 % of $\omega_{max} \approx 0.24$ rad/s.

1. Setup and experiment pipeline

Hardware (as found on the bench)

- Arduino Mega 2560 on /dev/ttyACM0, 115200 baud. The lab's original sketch was backed up (firmware/backup/original_lab_sketch.hex) and its wiring decoded from the binary: two-PWM motor driver (pin 4 = positive, pin 5 = negative direction), quadrature encoder on pins 18/19, 2150 counts/rev (x1).
- Our firmware (firmware/motor_io/motor_io.ino) is a thin real-time I/O layer: every 10 ms it sends time, angle, velocity, potentiometer and applied PWM; it applies the PWM sent back by Python and switches the motor off if no command arrives for 200 ms. Encoder decoded x4 (8600 counts/rev), velocity low-pass filtered (α = 0.5). Measured sample period: 10.00 ms, max 10.01 ms.
- The motor is a slow geared motor: about 4.7 rad/s (≈ 45 rpm) at full PWM.

Software

- All controllers run in Python (lab/controllers.py) in a hardware-in-the-loop configuration; one command per lab (python run.py lab1 … lab6) runs all experiments automatically and writes a report.
- A simulator (lab/rig.py, SimRig) reproduces the identified rig (dead time, stick-slip, quantisation); every controller was tuned on it before running on the motors.
- Safety: abort on |ω| > 20 rad/s, on adaptive parameter divergence (|θ| > 20 normalised), and, added after the Lab I.6 fault, on full PWM without measured motion for 2 s.

2. Lab I.1: modelling and identification

Model. First-order plus dead time with stick-slip friction: τ dω/dt = −ω + K(u(t − Td) − u0), valid while the shaft moves; from rest the shaft only starts when |u| exceeds the breakaway PWM ub, and a moving shaft sticks again when |u| drops below the stop threshold us.

Step tests (open loop, from rest)

u [PWM]	ω_ss [rad/s]	Td [s]	τ [s]	rise 10–90 % [s]
30	0	–	–	–
50	0	–	–	–
75	0	–	–	–
100	0	–	–	–
130	2.47	0.07	0.19	0.38
160	3.19	0.1	0.13	0.3
200	3.83	0.11	0.12	0.27
255	4.74	0.12	0.09	0.2

u [PWM]	ω_ss [rad/s]	Td [s]	τ [s]	rise 10–90 % [s]
-30	0	–	–	–
-50	0	–	–	–
-75	0	–	–	–
-100	0	–	–	–
-130	-2.83	0.08	0.17	0.39
-160	-3.48	0.12	0.14	0.38
-200	-4.04	0.12	0.12	0.29
-255	-4.74	0.13	0.09	0.22

Steps of |u| ≤ 100 PWM (up to 100 for u < 0) did not break the static friction: ω_ss = 0.

Identified parameters

	u > 0	u < 0	used for design
gain K (kinetic line, slow ramp) [rad/s/PWM]	0.0210	0.0186	0.0198
gain K from steps (cross-check)	0.0178	0.0149	
dead zone of the kinetic line u0 [PWM]	22	0	
breakaway from rest ub: steps / slow ramp [PWM]	115 / 153	115 / 181	varies per start: 87–200

	u > 0	u < 0	used for design
stop threshold us [PWM]	75	60	
time constant τ [s]	0.125	0.130	0.128 (a = 7.84 s^{-1})
dead time Td [s]			0.115
ω_max [rad/s]			4.74

The parameters were repeatable across three identification runs (τ 0.127–0.13 s, Td 0.105–0.115 s, K 0.017–0.020). The breakaway PWM is not: it changed between 87 and 200 PWM from one start to the next (depends on shaft/gear position). rise(10–90 %) ≈ 0.27–0.38 s vs 2.2·τ = 0.28 s is consistent with first order.

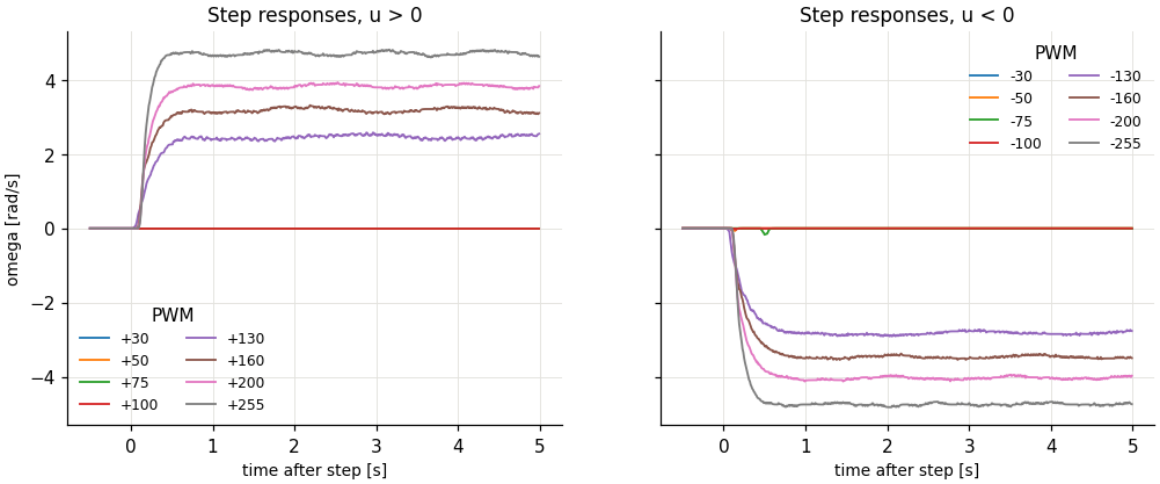


Figure 1: open-loop step responses (all levels).

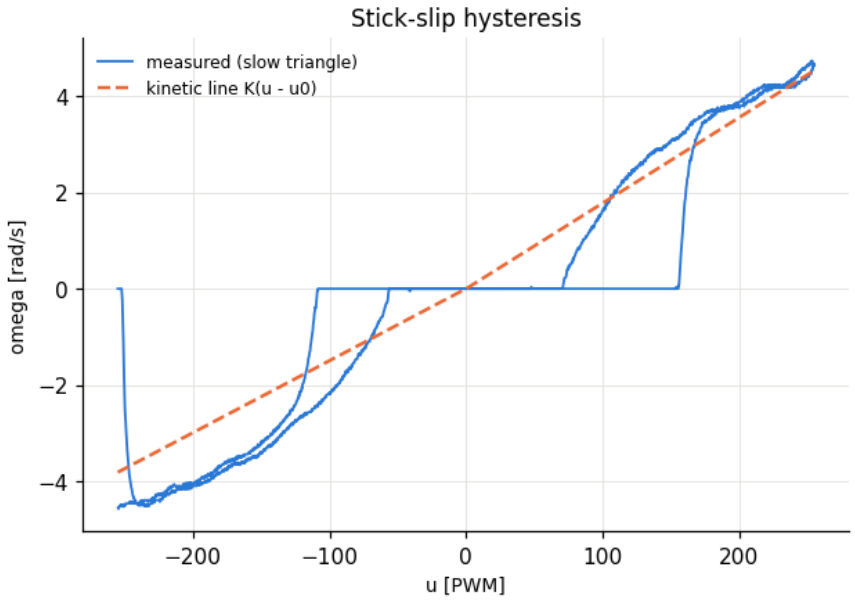


Figure 2: slow PWM triangle: stick-slip hysteresis. The shaft needs much more PWM to start than to keep moving; the moving branches define the kinetic line K(u − u0).

Model validation

input	fit % (all samples)	fit % while moving	RMS error [rad/s]
step 153 PWM	74.8	n/a (constant speed)	0.27
slow sine ±179 PWM, 10 s	51.4	85.1	1.09
ramp/triangle ±204 PWM, 12 s	71.1	83.2	0.65

While the shaft moves, the model reproduces the response well (83–85 %). The overall fit is lower because the breakaway instants are random and cannot be predicted by any deterministic model

(Figure 3).

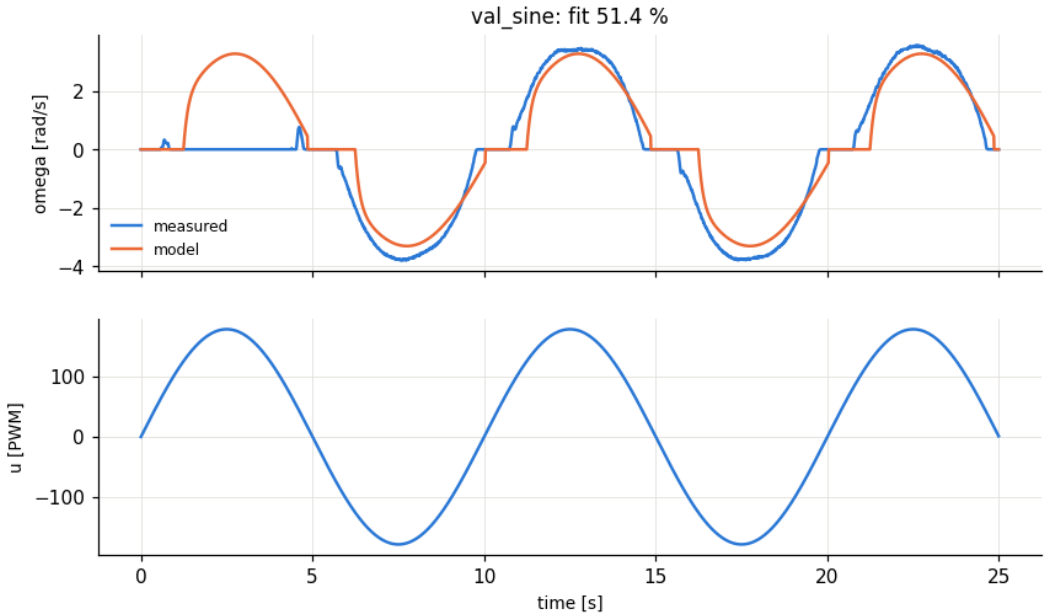


Figure 3: validation with a slow sinusoidal PWM (measured vs model).

P and PI control from the model

Because of the dead time the closed loop cannot be made much faster than Td, so the controllers were tuned with the lambda (IMC) rule for a FOPDT plant: $\lambda = \max(\tau/3, 2 \cdot Td) = 0.23$ s, $Kp = \tau/(K(\lambda + Td))$, $Ti = \tau$. (A first design that ignored the dead time, Kp = 158, produced a stick-slip limit cycle.)

controller	gains	predicted ω/r	measured ω/r	RMS error [rad/s]
P	Kp = 18.7	0.27	0 (shaft never started)	2.01
PI	Kp = 18.7, Ki = 146.5	1	0.998	0.58

The P controller never moves the shaft: Kp·e stays far below the breakaway PWM. The PI integrator builds up the missing PWM and tracks every level with zero steady-state error; at each sign change the shaft stalls for ~0.6 s while the integrator crosses the stiction band (Figure 4).

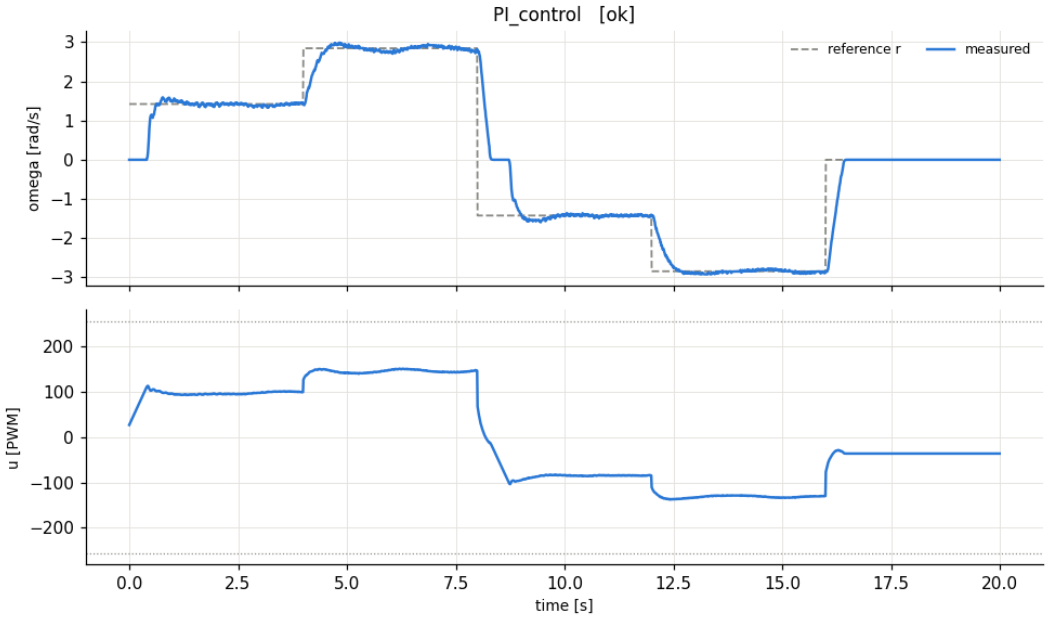


Figure 4: PI velocity control on reference steps.

Effect of the load potentiometer on the second motor

load	u [PWM]	ω_ss [rad/s]	gain ω/u	τ [s]	a = 1/τ [s⁻¹]
minimum	+180	3.59	0.0199	0.135	7.4
minimum	-180	-3.84	0.0213	0.135	7.4
middle	+180	3.55	0.0197	0.135	7.4
middle	-180	-3.78	0.0210	0.125	8.0
maximum	+180	0.80	0.0045	0.035	28.6
maximum	-180	-0.97	0.0054	0.045	22.2

Maximum load cuts the steady-state gain by ≈ 4× (−75 to −78 %) and makes the pole ≈ 3.5× faster: the generator's braking torque (∝ speed) adds damping. The repeat scatter is ±0.02 rad/s, so this is > 100× the noise. Between the minimum end and the middle the effect is ≤ 2 % (not significant): braking current ∝ $1/(R_{armature} + R_{pot})$ only becomes large when R_{pot} approaches the armature resistance, near the maximum end. The pot was initially not connected (no effect at any position); two runs affected by a wrong pot position and a loose contact were excluded (results/rig/load_test/excluded/). The identification above was done at minimum/middle load.

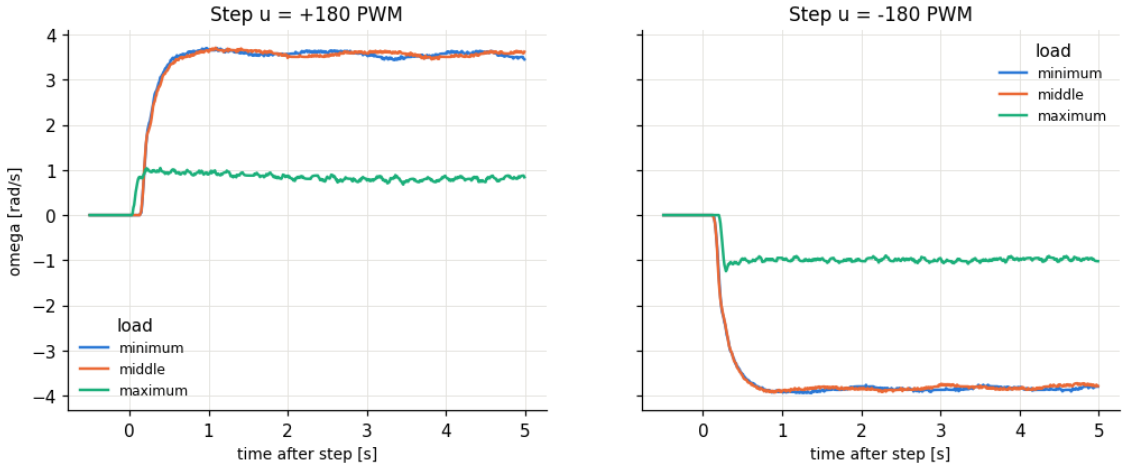


Figure 5: step responses for minimum, middle and maximum load.

3. Adaptive control: common design (Labs I.2–I.6)

- Normalised plant: $dy/dt = -a \cdot y + b_n \cdot u_n$, $a = 7.84\ s^{-1}$, $b_n = 8.35$.
- Reference model: $dy_m/dt = -a_m y_m + b_m r$ with $a_m = b_m = \min(2a, 1/(3 \cdot Td)) = 2.9\ s^{-1}$ (time constant 0.35 s). The dead time makes a faster model impossible to follow.
- References: steps of ±0.3 and ±0.6 ω_{max} (5 s each), slow sine (amplitude 0.5, period 10 s, as in the manual), a faster sine (period 3 s) for comparing persistent excitation (PE), and a long constant step at 0.5 ω_{max}.
- PE level = worst $\lambda_{min}/\lambda_{max}$ of the windowed regressor covariance (0 = no excitation).
- The "ideal" parameter values printed in the tables follow from the linear model; with stiction the controllers settle elsewhere, because the real plant has less gain at low speed.
- The verdict "poor tracking" in the reports means RMS error in the last 25 % above 5 % of ω_{max}; on this rig it is almost always caused by stick-slip at zero crossings, not by instability.

4. Lab I.2: MIT rule

$u = \theta_1 r - \theta_2 y$, $d\theta_1/dt = -\gamma[a_m/(s+a_m)\ r]e$, $d\theta_2/dt = \gamma[a_m/(s+a_m)\ y]e$, $\gamma = 5$ for the tracking runs.

experiment	RMS e	RMS e last 25 %	PE level	final θ1, θ2 (linear ideal 0.347, −0.592)
steps	0.127	0.071	0.004	1.32, 0.44
slow sine	0.122	0.103	0.006	1.41, 0.10
fast sine	0.137	0.124	0.073	0.92, −0.06

Stability map (γ × reference amplitude r0, square wave, 24 s per run)

γ \ r0	0.2	0.4	0.7
1	stuck (stiction), $\gamma r0^2 = 0.04$	stable (0.16)	stable (0.49)
3	stable (0.12)	stable (0.48)	stable (1.47)
10	stable (0.4)	stable (1.6)	stable (4.9)
30	stable (1.2)	stable (4.8)	stable (14.7), larger oscillation
100	stable (4)	UNSTABLE (16), aborted at 10 s	UNSTABLE (49), aborted at 0.9 s

Experimental stability condition: every stable run has $\gamma \cdot r0^2 \le 14.7$ and every unstable run $\gamma \cdot r0^2 \ge 16$, i.e. the MIT loop is stable for $\gamma \cdot r0^2 < c$ with $c \approx 15$. For the ideal first-order model the linearised MIT loop is second order and always stable; the dead time (≈ 0.12 s) is the unmodelled dynamics that creates the bound, and the bound scales with the square of the signal level, as predicted. The boundary rests on one row (γ = 100); a finer sweep between γ = 30 and 100 would locate c more precisely.

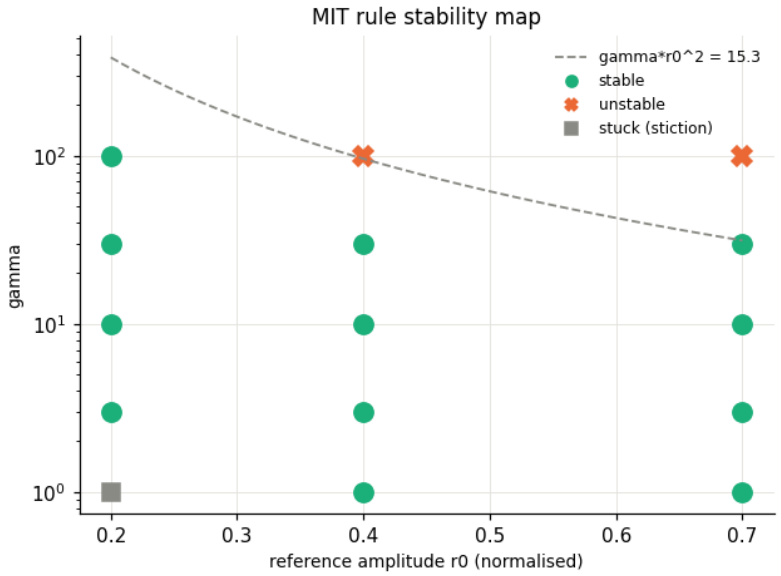


Figure 6: MIT rule stability map; dashed line $\gamma \cdot r0^2 = 15.3$.

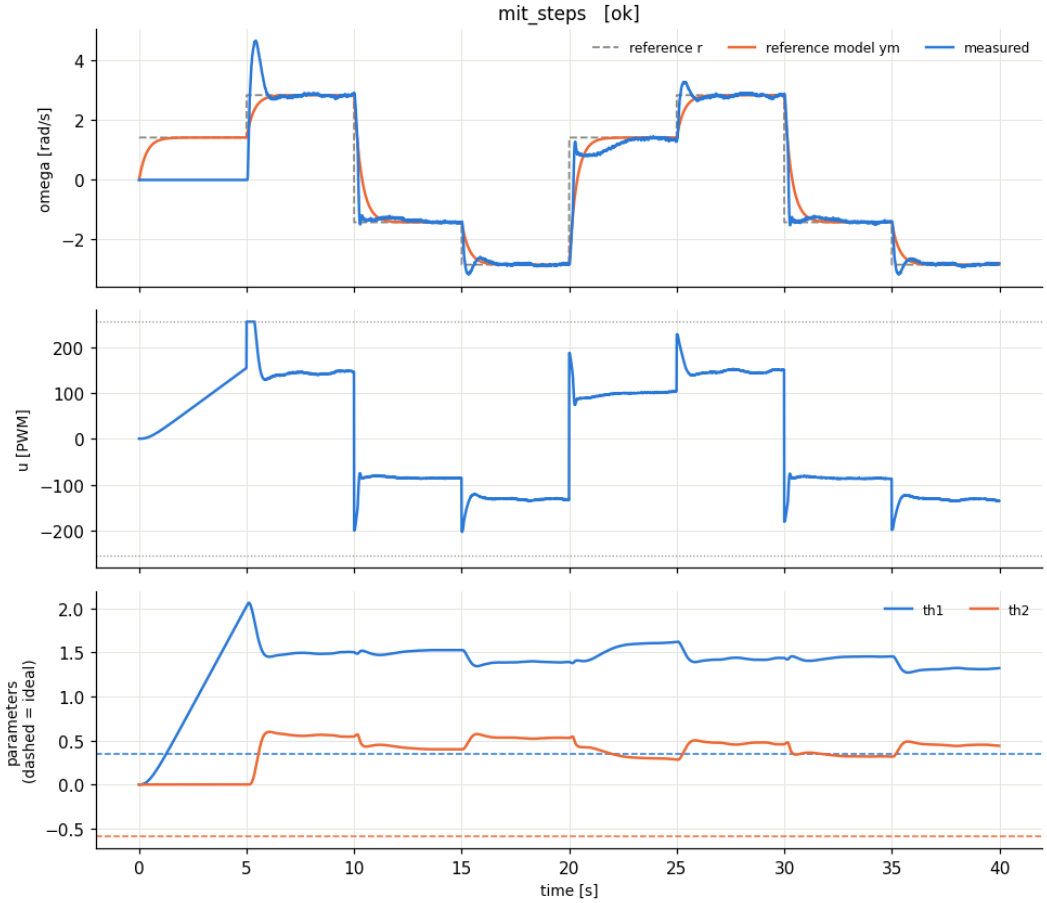


Figure 7: MIT rule on reference steps ($\gamma = 5$).

5. Lab I.3: scalar MRAC

$u = k \cdot y + l \cdot r$, $dk/dt = -\gamma \cdot y \cdot e$, $dl/dt = -\gamma \cdot r \cdot e$, $\gamma = 3$ ($\gamma \geq 20$ was unstable in simulation). Initial estimates 0.

experiment	RMS e	RMS e last 25 %	PE level	drift (2nd half)	final k, l (linear ideal 0.592, 0.347)
steps	0.154	0.070	0.015	1.11	−0.89, 1.74
slow sine	0.124	0.099	0.015	0.31	−0.22, 1.27

experiment	RMS e	RMS e last 25 %	PE level	drift (2nd half)	final k, l (linear ideal 0.592, 0.347)
fast sine	0.139	0.118	0.073	0.17	−0.24, 1.10
constant step, 10 min	0.027	0.015	0.0002	0.12	−0.21, 1.22

- **Long run:** no instability in 10 min (speed held at 2.4 rad/s, 1.5 % RMS error, input flat at ≈ 135 PWM), but k and l drift linearly in opposite directions (l 1.0 → 1.23, k 0.0 → −0.21) without saturating. On a constant reference y ≈ r, so only the combination k·y + l·r is identifiable (it stays constant); noise moves the parameters along the unidentifiable direction. This is the parameter drift that the robust modifications of Lab I.5 address.
- **Stiction:** at the small level (1.4 rad/s) the shaft sticks and the adaptive law integrates the error, so l winds up to 2.2 until the shaft breaks free, followed by an overshoot to 4.4 rad/s.
- **PE:** the slow sine (manual) is weakly exciting (0.015); the fast sine raises it to 0.073.

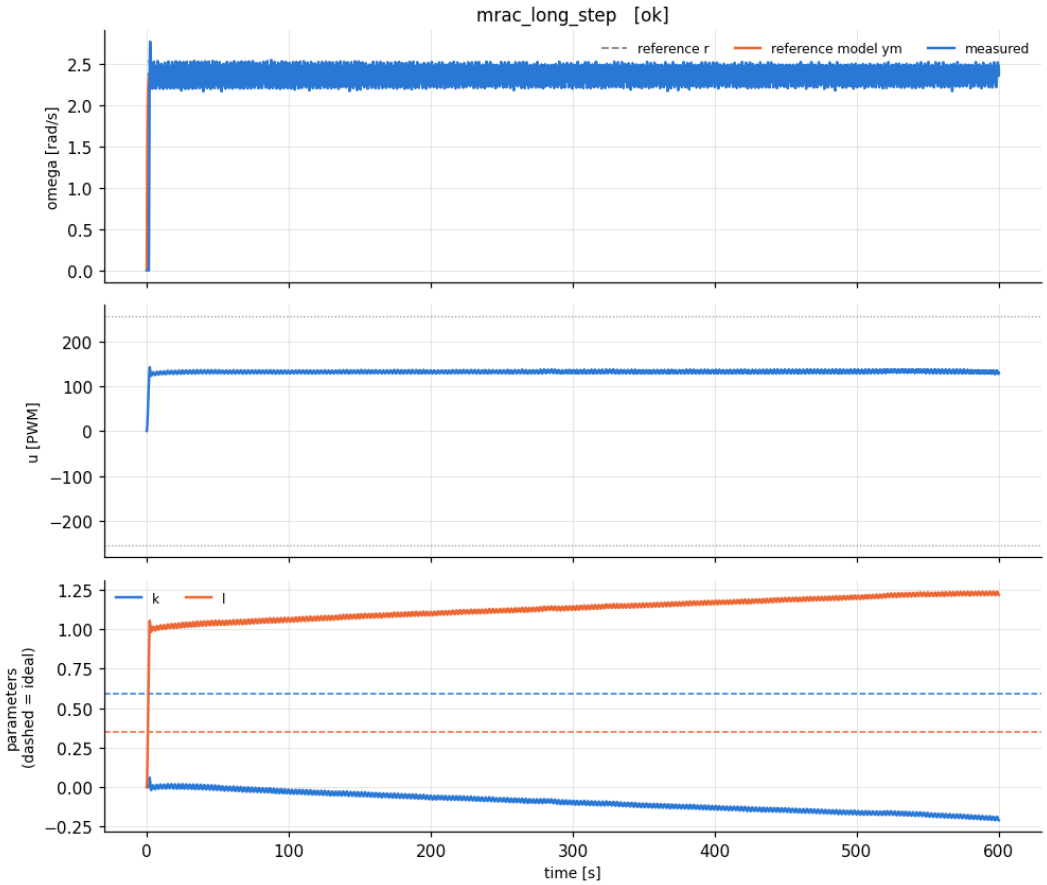


Figure 8: MRAC, 10 min constant step: stable tracking, linear parameter drift.

6. Lab I.4: scalar adaptive dynamic inversion (ADI)

Plant estimate $dy/dt = \hat{a}\cdot y + \hat{b}\cdot u$; control $u = (dy_m/dt - k_e e - \hat{a}\cdot y)/\hat{b}$; $d\hat{a}/dt = \gamma\cdot y\cdot e$, $d\hat{b}/dt = \gamma\cdot u\cdot e$, $\hat{b} \geq 0.1\, b_n$; $\gamma = 3$, $k_e = 2\ s^{-1}$ ($k_e \lesssim 1/(3\ Td)$). Initial estimates 50 % of the model values. (“ADI” was interpreted as adaptive dynamic inversion, the indirect counterpart of MRAC.)

experiment	RMS e	RMS e last 25 %	PE level	drift	final â, b̂ (linear ideal −7.84, 8.35)
steps	0.234	0.105	0.052	1.13	−1.54, 1.90
slow sine	0.255	0.292	0.037	2.16	−1.52, 1.58
fast sine	0.154	0.143	0.093	0.40	−3.19, 3.33
constant step, 10 min	0.055	0.020	0.0003	0.21	−2.10, 2.09

- On the slow sine most positive half-periods never break free: the ADI output peaks at 70–130 PWM, below the breakaway PWM; ADI has no integrator-like term to build the missing PWM (MRAC's l effectively did that).
- Long run: stable (2 % error); â and b̂ drift together (−2.5 → −2.1 and 2.5 → 2.1 in 10 min) along the unidentifiable direction â·y + b̂·u = const, slightly slower than MRAC.
- The estimates settle far below the linear model (b̂ 1.6–3.3 vs 8.35): with stiction the motor responds less to small inputs, and a smaller b̂ makes the inversion more aggressive.

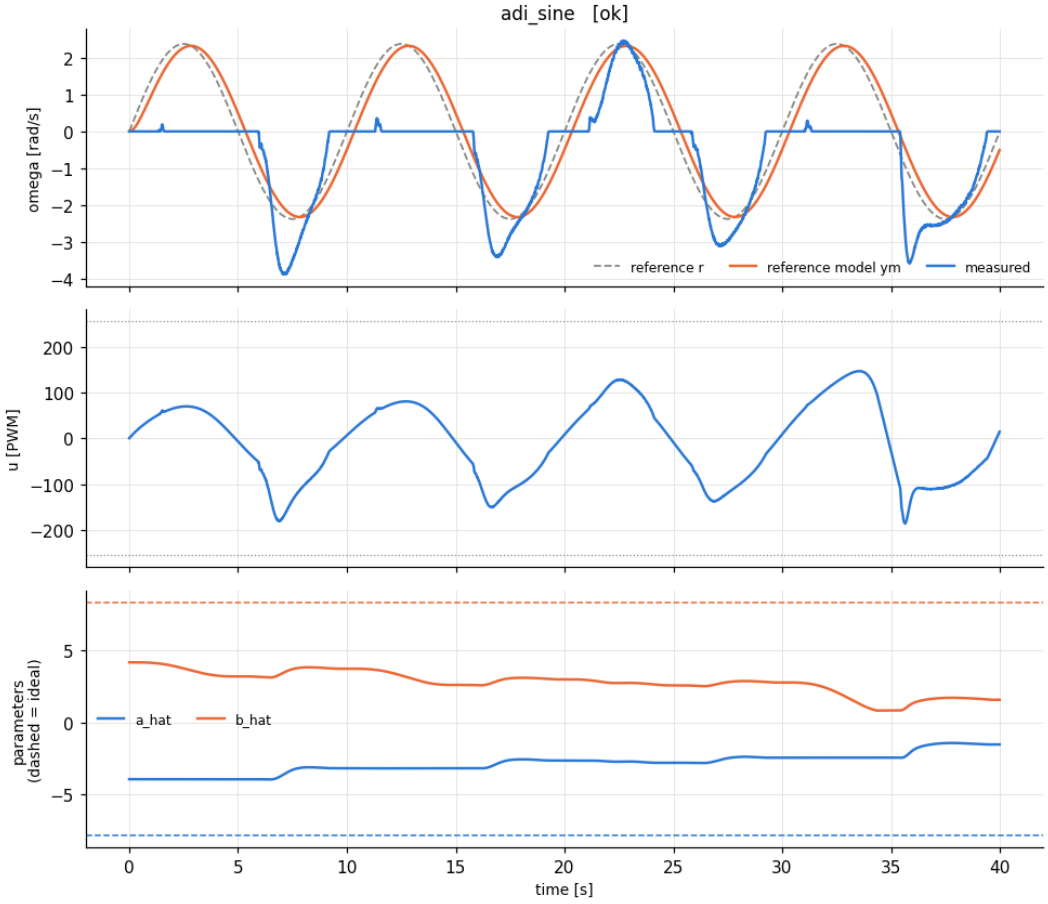


Figure 9: ADI on the slow sine: stuck positive half-periods.

7. Lab I.5: robust adaptive control (scalar MRAC and ADI)

Uncertainty model: offset + Coulomb friction, basis [1, tanh(v/v0)]. Design choices made after simulation tests (a naive version was worse than plain MRAC): (i) friction term seeded with the measured stop threshold (≈ 67 PWM) instead of 0; (ii) basis driven by the reference-model velocity y_m, so it is non-zero at standstill and helps break stiction (known signal, Lyapunov analysis unchanged); (iii) offset/friction terms adapt 0.3× slower; (iv) σ/e-mod leak toward the nominal Lab I.1 values. Robust settings: deadzone e0 = 0.02, σ = 0.01, projection |θi| ≤ 5. Each method: steps 40 s, slow sine 40 s, constant step 5 min.

controller	steps, e last 25 %	slow sine, e last 25 %	long step
MRAC (I.3)	7.0 %	9.9 %	l drift +0.23 / 10 min
MRAC + friction model, no modification (I.5)	4.3 %	4.6 %	practically no drift
ADI (I.4)	10.5 %	29 %	slow drift
ADI + friction model, no modification (I.5)	3.8 %	6.3 %	steady drift (Fig. 11)

method	MRAC steps	MRAC sine	MRAC long	ADI steps	ADI sine	ADI long
none	4.3 %	4.6 %	1.2 %	3.8 %	6.3 %	2.6 %
deadzone	4.0 %	4.7 %	1.6 %	4.4 %	6.9 %	2.5 %
σ-modification	5.3 %	6.2 %	2.4 %	4.4 %	9.0 %	4.0 %
e-modification	3.9 %	4.8 %	1.2 %	4.7 %	7.3 %	2.6 %
projection	3.6 %	4.8 %	1.2 %	4.2 %	6.9 %	2.3 %

(RMS normalised error over the last 25 % of each run.)

- With friction modelled, the friction estimate settles at a consistent level (MRAC dc ≈ 0.26–0.31, i.e. ≈ 70 PWM, matching the measured stop threshold) and the MRAC feedforward gain lands near the linear-model value (I ≈ 0.35–0.39 vs 0.347), instead of 1.2–1.7 in Lab I.3.
- **Drift (long constant step):** ADI without modification, and with deadzone, e-mod or projection, shows a steady linear decrease of ‖θ‖ (5.8 → 5.5 in 5 min). Only σ-modification stops it (plateau at 5.74), at the cost of a larger error (4.0 % vs 2.5 %): the classical leakage trade-off.
- The deadzone did not stop the drift because e0 = 0.02 is smaller than the velocity noise on this rig (|e| ≈ 0.03–0.05): adaptation never switches off. e0 ≈ 0.06 would be appropriate.
- Projection never activated: the parameters stayed well inside the bound |θi| ≤ 5.
- MRAC barely drifted with any method; σ-mod moved it to a new biased equilibrium (‖θ‖ 0.45 → 0.6, error ≈ 2×).
- PE on the slow sine is very low (0.001–0.004) for these 4-parameter models: the slow sine is not sufficiently rich to identify them.

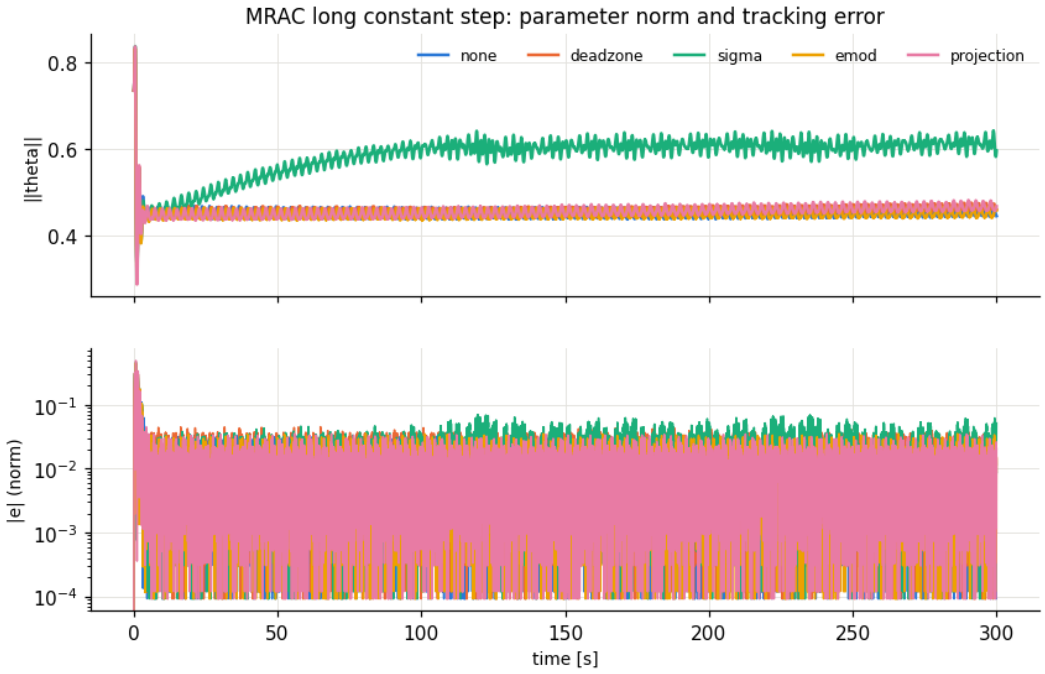


Figure 10: MRAC, 5 min constant step: parameter norm and |e| per method.

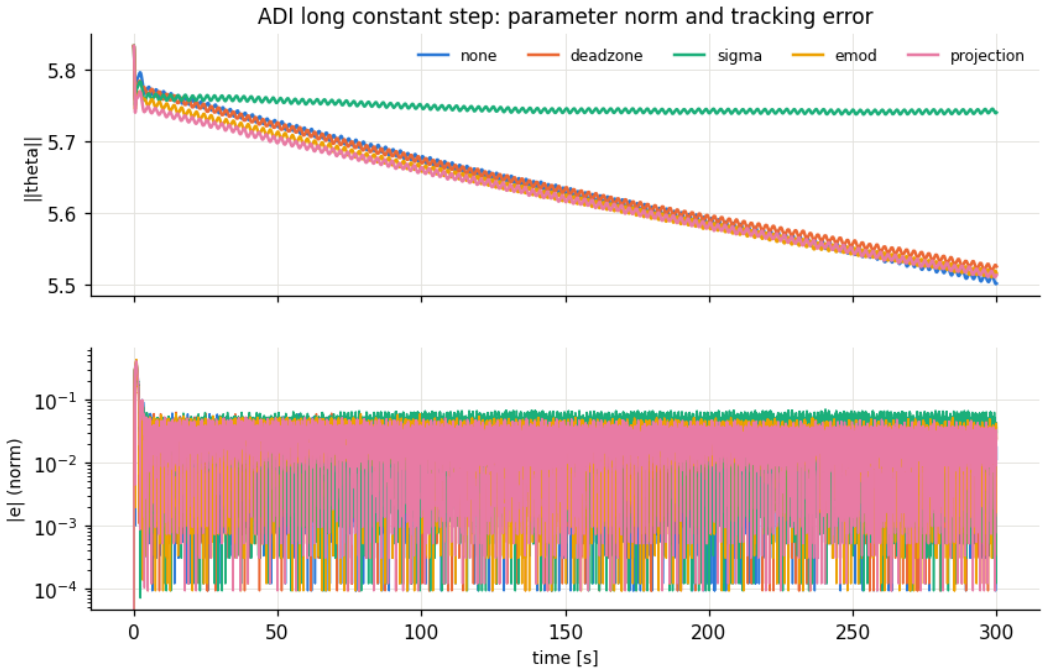


Figure 11: ADI, 5 min constant step: drift for all methods except σ-mod.

8. Lab I.6: robust adaptive angle control (partial)

x = [θ, ω]; reference model $\theta_m'' + 2\zeta\omega_n\theta_m' + \omega_n^2\theta_m = \omega_n^2 r$ with ω_n = 1.5 rad/s, ζ = 1 (limited by ω_{max} and the dead time); u = k1θ + k2ω + d0 + l·r plus a stiction compensation of 67 PWM at the output; dθ/dt = −Γw(e^TPB0), $A_m^T P + PA_m = -I$; γ = 5.

run	status	RMS angle error vs θm, last 25 %	final angle error
no modification, steps (±π, 2π)	valid	0.35 rad	0.23 rad
no modification, slow sine (±π)	valid	0.43 rad	(moving reference)
deadzone, steps	valid	0.47 rad	0.02 rad
deadzone, sine	fault at t = 18.9 s	–	–
σ-mod, e-mod, projection (steps, sine); 2 long steps	INVALID (motor failed)	–	–

The valid runs show that the angle controller works on this rig: the angle follows the reference model through steps of π and 2π, with errors mainly at the start of each step (stiction) and small hold errors (Figure 12).

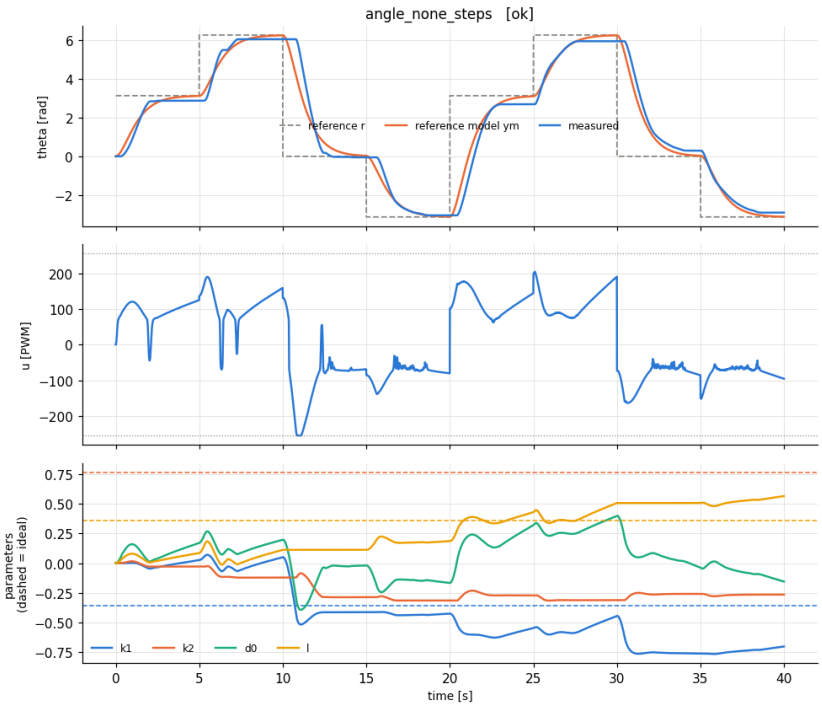


Figure 12: vector MRAC angle control on steps (valid run).

Hardware fault

At ≈ 16:21, 18.9 s into the deadzone/sine run, the encoder reading froze while the controller command was moderate (≈ 100 PWM); the controller then saturated at +255 PWM for the remaining ≈ 12 min, because the safety checks did not yet cover this case (a stall/sensor-loss watchdog was added afterwards). A full-PWM test and an encoder diagnostic sketch afterwards measured zero encoder edges on both channels in both directions; on inspection the gears move but the motor does not, so the motor is most likely broken. The failure occurred at a moderate command; whether the following 12 min at full PWM contributed to the damage cannot be determined.

9. Not done / to do next session

- **Repair or replace the motor**, then run `python run.py check`.
- **Re-run Lab I.6 completely** (≈ 17 min): all five robust methods on steps and sine, and the two 5 min long steps (none vs projection) for the drift/instability question. Because a new motor may differ, re-run the Lab I.1 identification first (≈ 5 min) so the model and the stiction compensation match.
- If a new motor is fitted, Labs I.2–I.5 results describe the old motor; repeating at least I.3 would show whether the conclusions carry over.
- Optional improvements suggested by the data: Lab I.5 deadzone width e0 ≈ 0.06 (above the noise), so the deadzone actually stops the drift; finer MIT sweep between γ = 30 and 100 to locate c; a longer (≥ 30 min) MRAC run to see whether the drift eventually causes bursting.
- Part II (coupled water tanks) is outside the scope of this session.

Files

- Reports, plots and CSV data: `results/rig/<lab>_<time>/`; identified model: `results/rig/model.json`; load test: `results/rig/load_test/`. Code: `run.py`, `lab/`, `experiments/`, firmware in `firmware/motor_io/`; all settings in `lab/config.py`.